UMGC
Accommodation Reservation
Software Design Document

Name: Alora Wilson
Lab Section:
Workstation:
Date: (02/22/2026)

Table of Contents

1 INTRODUCTION 3
1.1 Purpose 3
1.2 Scope 3
1.3 Overview 3
1.4 Reference Material 4
1.5 Definitions and Acronyms 4
2 SYSTEM OVERVIEW 4
3 SYSTEM ARCHITECTURE 5
3.1 Architectural Design 5
3.2 Decomposition Description 5
3.3 Exception Handling 6
3.4 Design Rationale 8
4 DATA DESIGN 9
4.1 Data Description 9
4.2 Data Dictionary 11
5 COMPONENT DESIGN 14
public double calculateTotalPrice() 15
6 HUMAN INTERFACE DESIGN 32
6.1 Overview of User Interface 32
6.2 Screen Images 35
6.3 Screen Objects and Actions 40
Back Button 42
Save Changes Button 44
Cancel Button 44
7 REQUIREMENTS MATRIX 44

1 INTRODUCTION

1.1 Purpose

This Software Design Document (SDD) describes the architecture, design components, data structures, and user interaction model of the Accommodation Reservation Management System. The purpose of this document is to provide a detailed technical blueprint of the system so that developers, testers, and technical stakeholders can understand how the system is structured and how it should be implemented.

This document serves as a reference for implementation, validation, maintenance, and future enhancements of the system.

1.2 Scope

The system manages customer accounts and lodging reservations (Hotel, Cabin, and House). It allows users to:

- Create and manage accounts
- Add, update, complete, and cancel reservations
- Calculate reservation pricing
- Enforce lifecycle validation rules
- Handle exceptions for invalid operations

The current implementation manages customer accounts and reservations in memory using Java collections. File-based persistence is identified as a planned enhancement and is not implemented in the current version. No relational database is used.

Functionality outside the scope of this design includes:

- Graphical interface implementation
- Database integration
- Payment processing
- External API integrations

.

1.3 Overview

This document is organized into the following sections:

- Section 2 describes the overall system functionality.
- Section 3 describes the system architecture and design decomposition.
- Section 4 defines data structures and file formats.
- Section 5 provides detailed component-level design including constructors and method pseudocode.
- Section 6 describes the user interface design and user interaction.
- Section 7 provides a requirements matrix mapping requirements to implementation components.

1.4 Reference Material

- Course examples
- Course provided SDD document template

1.5 Definitions and Acronyms

N/A

2 SYSTEM OVERVIEW

The Accommodation Reservation Management System is an object-oriented Java application designed to manage customer accounts and lodging reservations. The system supports three types of reservations: Hotel, Cabin, and House. Each reservation type extends a common abstract Reservation base class and implements its own pricing logic.

The system is structured around a central Manager class, which serves as the coordination layer between the user interface and the domain model. The Manager maintains a memory collection of Account objects. Each Account object contains customer information and a list of associated Reservation objects.

The system provides the following major functionality:

- Create new customer accounts
- Retrieve and update account information
- Add reservations to an account
- Complete reservations and calculate final pricing
- Cancel reservations
- Update reservation details (subject to lifecycle validation rules)
- Enforce business rules through exception handling

All pricing calculations are determined by reservation type and property characteristics such as beds, bedrooms, bathrooms, square footage, and special features (kitchenette, loft, or full kitchen).

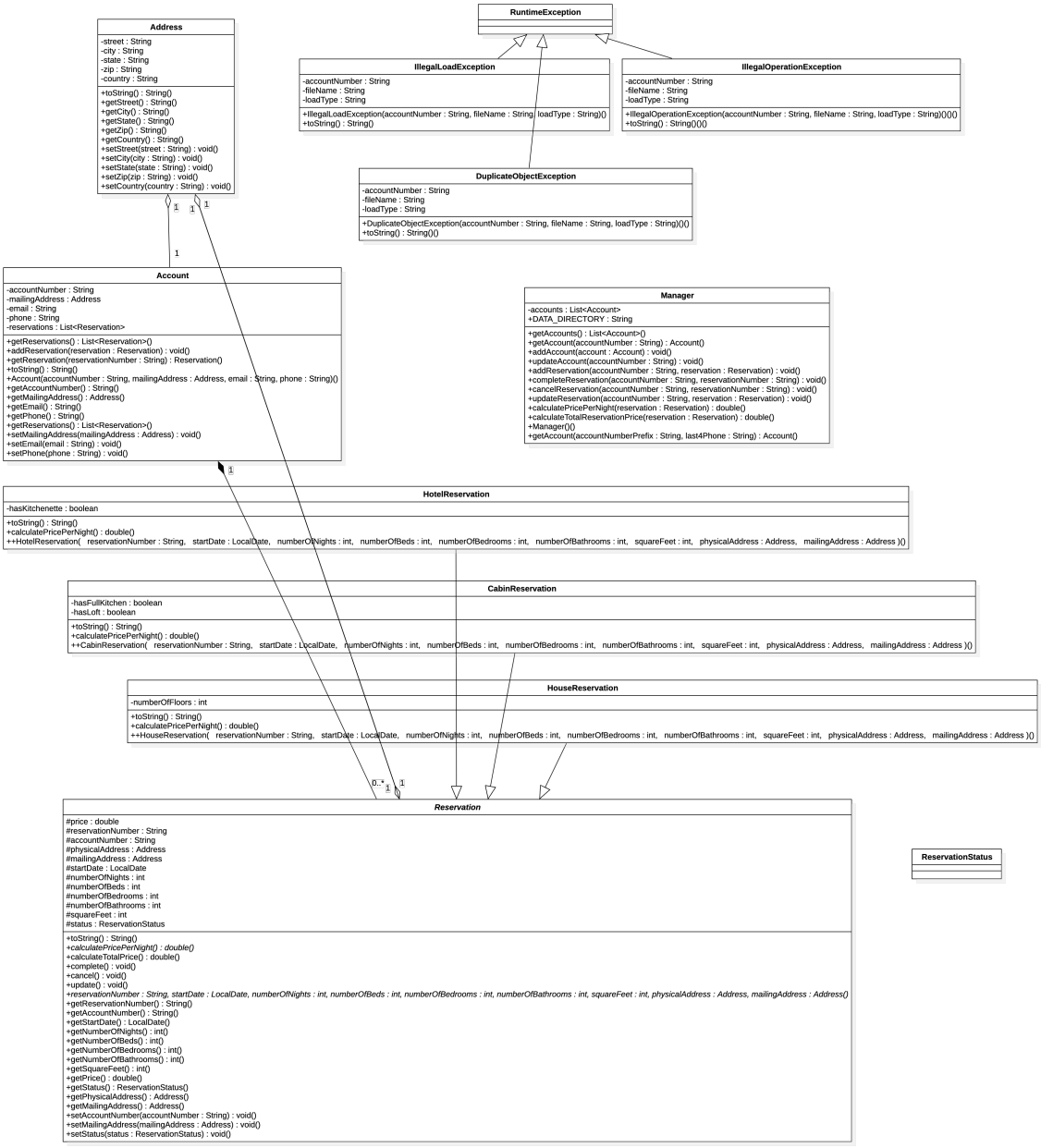
The system enforces reservation lifecycle states using the ReservationStatus enumeration. A reservation may exist in one of three states: DRAFT, COMPLETED, or CANCELLED. Lifecycle rules restrict which operations may be performed in each state.

The current implementation maintains customer accounts and reservations in memory using Java collections. The Manager class coordinates account and reservation operations but does not currently provide persistent storage. File-based persistence is planned as a future enhancement, with IllegalLoadException defined to support error handling during future file-loading operations. No relational database or external storage system is used.

The system is implemented in Java and follows an object-oriented design model emphasizing inheritance, encapsulation, and controlled access to data through getters, setters, and lifecycle validation methods.

3 SYSTEM ARCHITECTURE

3.1 Architectural Design



3.2 Decomposition Description

When the system starts, the user interface creates an instance of the Manager class. The Manager serves as the central coordination component of the system and maintains the collection of all Account objects in memory.

When the system starts, the Manager initializes an empty collection of Account objects in memory. Accounts and their associated reservations are created and

managed during application execution. The Manager does not currently load account or reservation data from files. File-based persistence is a planned enhancement that would allow the system to restore previously saved accounts and reservations. The IllegalLoadException class has been defined to support error handling when this functionality is implemented.

Once initialized, the UI interacts only with the Manager class. The UI does not directly manipulate Account or Reservation objects. All requests such as creating an account, adding a reservation, completing a reservation, or cancelling a reservation are routed through Manager methods.

When creating a reservation, the UI first constructs a specific reservation type (HotelReservation, CabinReservation, or HouseReservation). The Manager validates the request, assigns the reservation to the appropriate Account, and sets the reservation's account number. Duplicate detection is handled before the reservation is added to the Account's reservation list.

The Account class manages its internal collection of reservations. It provides methods to retrieve reservations by reservation number and ensures that duplicate reservations are not stored.

The abstract Reservation class defines shared data and lifecycle behavior. Subclasses implement the calculatePricePerNight() method to determine pricing logic specific to the reservation type. The total reservation price is calculated only when the reservation is completed. Lifecycle methods (complete, cancel, and update) enforce business rules by validating the current state before allowing changes.

The ReservationStatus enumeration defines the valid states of a reservation and is used throughout the system to enforce lifecycle constraints.

Exception classes are used to maintain system integrity. IllegalOperationException is thrown when lifecycle rules are violated. DuplicateObjectException is thrown when duplicate accounts or reservations are detected. IllegalLoadException is defined to support error handling for planned file-loading functionality but is not currently used by the application.

3.3 Exception Handling

The system uses custom runtime exceptions to enforce business rules and maintain data integrity. These exceptions provide detailed contextual information to assist in debugging and user feedback.

IllegalOperationException

This exception is thrown when a reservation lifecycle rule is violated. It is used within the Reservation class in the following methods:

- complete()
- cancel()
- update()

An IllegalOperationException is thrown when:
- A cancelled reservation is completed.
- A completed reservation is cancelled.
- A reservation is updated after completion or cancellation.
- A reservation is cancelled after its start date.
- A reservation is completed when not in DRAFT status.

The exception message format is:
"IllegalOperationException: Cannot '<operation>' reservation '<reservationNumber>' for account '<accountNumber>'. Reason: <reason>"
The toString() method returns the generated message using getMessage().

DuplicateObjectException

This exception is thrown when an attempt is made to add a duplicate object to the system. It is used in:
- Manager.addAccount() (duplicate account number)
- Manager.addReservation() (duplicate reservation number within an account)
- Account.addReservation() (duplicate reservation within account)

Two message formats are supported:

For duplicate account:
"DuplicateObjectException: Account with number '<accountNumber>' already exists."

For duplicate reservation:
"DuplicateObjectException: Reservation '<reservationNumber>' already exists for account '<accountNumber>'."

The toString() method returns the message via getMessage().

IllegalLoadException

The IllegalLoadException class is defined to support error handling for future file-based persistence functionality. Although the exception class exists in the current Java implementation, the Manager class does not yet perform file-loading operations or throw this exception.

Once file persistence is implemented, this exception is intended to handle:

- Failures when loading account files.
- Failures when loading reservation files.

The exception message format is:
"IllegalLoadException: Failed to load <loadType> file '<fileName>' for account '<accountNumber>'."

The toString() method returns the message via getMessage().

Standard Java Exceptions
The system also uses IllegalArgumentException for input validation errors, including:

- Null or empty account numbers
- Null reservation objects
- Invalid method parameters
- Account or reservation not found during lookup

3.4 Design Rationale

The system was designed using an object-oriented structure to clearly separate responsibilities between components and reduce tight coupling between classes.

The Manager class was designed as the central coordination layer so that the UI does not directly manipulate Account or Reservation objects. This ensures that validation rules and business logic are consistently enforced in one place. By routing all operations through the Manager, the system prevents uncontrolled state changes.

The Account class is responsible only for storing customer level data and managing its list of reservations. This keeps account data separate from system coordination logic.

The Reservation class was implemented as an abstract base class to avoid duplicating shared fields and lifecycle behavior across reservation types. Hotel, Cabin, and House reservations share common attributes (dates, beds, bathrooms, square footage, etc.), so placing those fields in the parent class reduces redundancy. Each subclass overrides calculatePricePerNight() so pricing logic can vary without affecting shared lifecycle behavior.

Lifecycle state management was implemented using the ReservationStatus enumeration. This makes valid states explicit and prevents the use of arbitrary string values. The complete, cancel, and update methods enforce state transitions directly within the Reservation class to guarantee data integrity.

Custom runtime exceptions were chosen to enforce business rules without requiring callers to explicitly catch checked exceptions. Each exception includes contextual information (account number, reservation number, operation name) so debugging and error reporting are clear and traceable.

File-based persistence was selected as a planned enhancement to keep the system lightweight and avoid the complexity of database integration. The current implementation uses in-memory Java collections to manage accounts and reservations. Future development will introduce text-file storage and loading capabilities, allowing account and reservation data to persist between application sessions.

4 DATA DESIGN

4.1 Data Description

The current implementation manages account and reservation data in memory using Java collections. The Manager class maintains a collection of Account objects, and each Account maintains its associated Reservation objects. Data is not currently saved between application sessions. File-based persistence is planned as a future enhancement, and the following file structures illustrate the proposed storage design.

In the proposed file-based persistence design, each account would be stored in a separate text file within the system's data directory. The proposed file naming convention uses the account number to uniquely identify each account file.

The following directory structure and file formats represent the proposed persistence design. They are included to document how account and reservation information could be stored and retrieved in a future implementation; these structures are not currently generated or accessed by the application.

Account files follow this naming format: Account-<accountNumber>.txt

For example: Account-1001.txt

Each account file contains account information (account number, mailing address, email, phone), a list of associated reservations, and a reservation date depending on reservation type. Reservation data is stored in structured tag format to allow identification of reservation type during loading.

Example:

<account>

  <accountNumber>1001</accountNumber>

```
<mailingAddress>
    <street>123 Main St</street>
    <city>Rockville</city>
    <state>MD</state>
    <zip>20850</zip>
    <country>USA</country>
</mailingAddress>
<email>customer@email.com</email>
<phone>555-123-4567</phone>

<hotelReservation>
    <reservationNumber>H001</reservationNumber>
    <startDate>2026-03-01</startDate>
    <numberOfNights>3</numberOfNights>
    <numberOfBeds>2</numberOfBeds>
    <numberOfBedrooms>1</numberOfBedrooms>
    <numberOfBathrooms>1</numberOfBathrooms>
    <squareFeet>450</squareFeet>
    <hasKitchenette>true</hasKitchenette>
    <status>DRAFT</status>
</hotelReservation>

<cabinReservation>
```

```
        <reservationNumber>C101</reservationNumber>
        <startDate>2026-06-10</startDate>
        <numberOfNights>5</numberOfNights>
        <numberOfBeds>3</numberOfBeds>
        <numberOfBedrooms>2</numberOfBedrooms>
        <numberOfBathrooms>2</numberOfBathrooms>
        <squareFeet>900</squareFeet>
        <hasLoft>true</hasLoft>
        <status>COMPLETED</status>
    </cabinReservation>
</account>
```

In the current implementation, the Manager class maintains account information in memory using Java collections. Reservation objects are associated with their respective Account objects and accessed through the application's existing methods. The system does not currently read or write account and reservation files. Future development will introduce file-loading and saving operations to support persistent storage between application sessions.

4.2 Data Dictionary

The following tables describes the primary data entities in the system.

Account –

Field	Type	Description
accountNumber	String	Unique identifier for account
mailingAddress	Address	Customer mailing address
email	String	Customer email address
phone	String	Customer phone number
reservations	List<Reservation>	Collection of reservations owned by the account

Address –

Field	Type	Description
reservationNumber	String	Unique reservation identifier
accountNumber	String	Associated account number
startDate	LocalDate	Start date of reservation
numberOfNights	int	Length of stay
numberOfBeds	int	Number of beds
numberOfBedrooms	int	Number of bedrooms
numberOfBathrooms	int	Number of bathrooms
squareFeet	int	Property size
price	double	Final calculated price
status	ReservationStatus	Lifecycle state

CabinReservation –

Field	Type	Description
hasLoft	boolean	Indicates whether the cabin includes a loft

DuplicateObjectException –

Field	Type	Description
accountNumber	String	Account involved in duplicate operation
reservationNumber	String	Reservation involved in duplicate operation

HotelReservation –

Field	Type	Description
hasKitchenette	Boolean	Indicates whether the hotel room includes a kitchenette

HouseReservation –

Field	Type	Description
hasFullKitchen	boolean	Indicates whether the house includes a full kitchen

IllegalLoadException –

Field	Type	Description
accountNumber	String	Account being loaded when failure occurred
fileName	String	File that failed to load
loadType	String	Indicates whether account or reservation load failed

IllegalOperationException –

Field	Type	Description
operation	String	Operation being attempted
accountNumber	String	Associated account
reservationNumber	String	Associated reservation
reason	String	Explanation of why operation failed

Manager –

Field	Type	Description
DATA_DIRECTORY	String	Directory path for stored data
accounts	List<Account>	Collection of accounts in memory

Reservation (abstract) –

Field	Type	Description
reservationNumber	String	Unique reservation identifier
accountNumber	String	Associated account number
startDate	LocalDate	Reservation start date
numberOfNights	int	Length of stay
numberOfBeds	int	Number of beds
numberOfBedrooms	int	Number of bedrooms
numberOfBathrooms	int	Number of bathrooms
squareFeet	int	Property size
price	double	Final calculated reservation price
status	ReservationStatus	Lifecycle state

ReservationStatus (Enum) –

Field	Description
CANCELLED	Reservation cancelled
COMPLETED	Reservation finalized
DRAFT	Reservation created but not finalized

5 COMPONENT DESIGN

Class Name: Reservation
Class Description/Purpose: The Reservation class is the abstract base class for all reservation types in the system (Hotel, Cabin, House). It stores the shared reservation data (dates, bed/bath counts, square footage, addresses, status, and price) and enforces the shared business rules for completing, canceling, and updating reservations. Subclasses provide the reservation-type-specific nightly pricing by implementing calculatePricePerNight().
Class Modifiers: public abstract
Class Inheritance: None (Base class)
Subclasses – HotelReservation, CabinReservation, HouseReservation

Class Attributes:

price – double – stores the finalized total price for the reservation. Price is only set when the reservation is completed
reservationNumber – string – unique reservation identifier generated by the UI layer
accountNumber – string – identifies the account that owns this reservation. Defaults to "-99" until assigned to an account
physicalAddress – Address – physical location of the lodging
mailingAddress – Address – mailing address for the lodging (may be the same as physical or different)
startDate – LocalDate – start date of the reservation
numberOfNights – int – length of stay in nights
numberOfBeds – int – number of beds for the reservation
numberOfBedrooms – int – number of bedrooms for the reservation
numberOfBathrooms – int – number of bathrooms in the reservation
squareFeet – int – approximate size of the lodging
status – ReservationStatus – reservation lifecycle state (DRAFT, COMPLETED, CANCELLED)

Exceptions Thrown: IllegalOperationException

This exception is thrown when a user attempts an action that violates reservation lifecycle rules (completing a cancelled reservation, cancelling a completed reservation, or updating a reservation that is not editable)

Class Constructors: There is one constructor used to initialize the shared reservation data. This constructor is called by subclass constructors to ensure all reservation types are created with the same required base fields. It also sets default values required by the system: accountNumber is initialized to "-99" and status is initialized to DRAFT.

public Reservation(String reservationNumber, LocalDate startDate, int numberOfNights, int numberOfBeds, int numberOfBedrooms, int numberOfBathrooms, int squareFeet, Address physicalAddress, Address mailingAddress)

Assign reservationNumber parameter to attribute reservationNumber
Assign startDate parameter to attribute startDate
Assign numberOfNights parameter to attribute numberOfNights
Assign numberOfBeds parameter to attribute numberOfBeds
Assign numberOfBedrooms parameter to attribute numberOfBedrooms
Assign numberOfBathrooms parameter to attribute numberOfBathrooms
Assign squareFeet parameter to attribute squareFeet
Assign physicalAddress parameter to attribute physicalAddress
Assign mailingAddress parameter to attribute mailingAddress
Set accountNumber attribute to "-99" (default until assigned to an Account)
Set status attribute to ReservationStatus.DRAFT

Class Methods:
public abstract double calculatePricePerNight()

This method is implemented by each subclass to compute the nightly rate for that specific reservation type.
Implemented in subclass
Return calculated nightly rate

public double calculateTotalPrice()

Total price is only considered finalized once the reservation has been completed. If the reservation is not completed, the method returns 0.
If status is not ReservationStatus.COMPLETED, return 0
Return price

public void complete()

Completes the reservation if allowed. Completion finalizes pricing by calculating the nightly rate and multiplying by number of nights. This method prevents completing reservations that are cancelled, already completed, or not in draft status.

If status is ReservationStatus.CANCELLED, throw IllegalOperationException("complete", accountNumber, reservationNumber, "Reservation is cancelled and cannot be completed.")

If status is ReservationStatus.COMPLETED, throw IllegalOperationException("complete", accountNumber, reservationNumber, "Reservation is already completed.")

If status is not ReservationStatus.DRAFT, throw IllegalOperationException("complete", accountNumber, reservationNumber, "Only draft reservations can be completed.")

perNight = calculatePricePerNight()

price = perNight * numberOfNights

status = ReservationStatus.COMPLETED

public void cancel()

Cancels the reservation if allowed. A completed reservation cannot be cancelled. A reservation can only be cancelled if its start date is in the future.

If status is ReservationStatus.COMPLETED, throw IllegalOperationException("cancel", accountNumber, reservationNumber, "Reservation is already completed and cannot be cancelled.")

If startDate is not null AND startDate is not after today, throw IllegalOperationException("cancel", accountNumber, reservationNumber, "Reservation start date is not in the future.")

status = ReservationStatus.CANCELLED

public void update()

Ensures the reservation is still editable. This method enforces update rules only; actual field changes are expected to be done through setter methods.

If status is ReservationStatus.CANCELLED, throw IllegalOperationException("update", accountNumber, reservationNumber, "Reservation is cancelled and cannot be updated.")

If status is ReservationStatus.COMPLETED, throw IllegalOperationException("update", accountNumber, reservationNumber, "Reservation is already completed and cannot be updated.")

If startDate is not null AND startDate is before today, throw IllegalOperationException("update", accountNumber, reservationNumber, "Reservation start date is in the past.")

(No direct updates occur here; setters handle changes.)

GETTER METHODS

Each getter returns the corresponding attribute value.

public String getReservationNumber()
Return reservationNumber

public String getAccountNumber()
Return accountNumber

public Address getPhysicalAddress()
Return physicalAddress

public Address getMailingAddress()
Return mailingAddress

public LocalDate getStartDate()
Return startDate

public int getNumberOfNights()
Return numberOfNights

public int getNumberOfBeds()
Return numberOfBeds

public int getNumberOfBedrooms()
Return numberOfBedrooms

public int getNumberOfBathrooms()
Return numberOfBathrooms

public int getSquareFeet()
Return squareFeet

public double getPrice()
Return price

public ReservationStatus getStatus()
Return status

SETTER METHODS

These methods allow limited changes after creation.

public void setAccountNumber(String accountNumber)
Assign accountNumber parameter to attribute accountNumber

public void setMailingAddress(Address mailingAddress)
Assign mailingAddress parameter to attribute mailingAddress

public void setStatus(ReservationStatus status)
Assign status parameter to attribute status

Class Name: Account
Class Description/Purpose: The Account class represents a customer account in the accommodation reservation system. It stores the customer's identifying account number, contact information (mailing address, email, phone), and the list of reservations that belong to the account. This class provides methods to add and retrieve reservations and to update contact information.
Class Modifiers: public
Class Inheritance: none
Class Attributes:

accountNumber – string – unique identifier for the account whose value is intended to remain unchanged once the account is created
mailingAddress – Address – the mailing address associated with the account
email – string – contact email for the account
phone – string – contact phone number for the account
reservations – List<Reservation> - a list storing reservations associated with the account. Initialized to empty ArrayList

Exceptions Thrown:
IllegalArgumentException – thrown when required values are missing or invalid (null account, null reservation, empty reservation number, etc)

DuplicateObjectException – thrown when attempting to add a reservation with a reservation number that already exists.

Class Constructors: The Account object is created when the system needs to store and manage customer account information. The constructor initializes the account's identifying data and prepares the reservations list so that reservations can be added immediately after creation.

public Account(String accountNumber, Address mailingAddress, String email, String phone)

Validate accountNumber is not null/empty (throw IllegalArgumentException if invalid)
Assign accountNumber parameter to attribute accountNumber
Assign mailingAddress parameter to attribute mailingAddress
Assign email parameter to attribute email
Assign phone parameter to attribute phone
Initialize reservations to a new empty ArrayList

Class Methods:

public List<Reservation> getReservations()
return reservations

public void addReservation(Reservation reservation)
For each existing reservation r in reservations:
If r.getReservationNumber() equals reservation.getReservationNumber():
Throw DuplicateObjectException(accountNumber,
reservation.getReservationNumber())
Add reservation to reservations

public Reservation getReservation(String reservationNumber)
Validate reservationNumber is not null/empty (throw IllegalArgumentException if invalid)
For each reservation r in reservations:
If r.getReservationNumber() equals reservationNumber:
Return r
Return null (if no matching reservation found)

public String toString()
Build a formatted String containing:
accountNumber
mailingAddress (or "N/A" if null)
email (or "N/A" if null)

phone (or “N/A” if null)
number of reservations in reservations
Return formatted

GET METHODS

Remain simple getters.

public String getAccountNumber()
Return accountNumber

public Address getMailingAddress()
Return mailingAddress

public String getEmail()
Return email

public String getPhone()
Return phone

SET METHODS

Allow updates to mutable contact info

public void setMailingAddress(Address mailingAddress)
Assign mailingAddress parameter to attribute mailingAddress

public void setEmail(String email)
Assign email parameter to attribute email

public void setPhone(String phone)
Assign phone parameter to attribute phone

Class Name: Manager
Class Description/Purpose: The Manager class is the central coordinating class for the system. It maintains the in-memory list of all Account objects and provides high-level operations for account management and reservation management (create, update, complete, cancel). The UI interacts with the Manager class rather than directly manipulating accounts and reservations. In later development, this class will also handle loading/saving account and reservation data to disk.
Class Modifiers: public

Class Inheritance: none
Class Attributes:

DATA_DIRECTORY – string (public static final) – directory path where account data will be stored (currently empty code string, placeholder for later)
accounts – List<Account> – list of accounts managed in memory. Initialized to an empty ArrayList

Exceptions Thrown:
IllegalArgumentException – thrown when required inputs are null or empty or when an account cannot be found for an exception

DuplicateObjectException – thrown when a duplicate account number or duplicate reservation number is detected

IllegalLoadException – intended for file-load failures during startup (documented in comments but not implemented yet)

Class Constructors: The Manager constructor initializes the system's account collection. When the system starts, the UI creates a Manager instance.

public Manager()
Initialize accounts to a new empty ArrayList (handled by field initialization)

Class Methods:

public List<Account>getAccounts()
return accounts

public Account getAccount(String accountNumber)
Validate accountNumber is not null or empty (throw IllegalArgumentException if invalid)
For each Account a in accounts:
If a.getAccountNumber() equals accountNumber:
Return a
Return null

public void addAccount(Account account)
Validate account is not null (throw IllegalArgumentException if null)
For each Account a in accounts:
If a.getAccountNumber() equals account.getAccountNumber():
Throw DuplicateObjectException(account.getAccountNumber())
Add account to accounts

public void updateAccount(String accountNumber)
Validate accountNumber is not null or empty (throw IllegalArgumentException if invalid)
Account account = getAccount(accountNumber)
If account is null:
Throw IllegalArgumentException("No account found for account number: " + accountNumber)

public void addReservation(String accountNumber, Reservation reservation)
Validate accountNumber is not null or empty (throw IllegalArgumentException if invalid)
Validate reservation is not null (throw IllegalArgumentException if null)
Account account = getAccount(accountNumber)
If account is null:
Throw IllegalArgumentException("No account found for account number: " + accountNumber)
For each Reservation r in account.getReservations():
If r.getReservationNumber() equals reservation.getReservationNumber():
Throw DuplicateObjectException(accountNumber, reservation.getReservationNumber())
reservation.setAccountNumber(accountNumber)
account.addReservation(reservation)

public void completeReservation(String accountNumber, String reservationNumber)
Validate accountNumber and reservationNumber are not null or empty (throw IllegalArgumentException if invalid)
Account account = getAccount(accountNumber)
If account is null:
Throw IllegalArgumentException("No account found for account number: " + accountNumber)
Reservation reservation = account.getReservation(reservationNumber)
If reservation is null:
Throw IllegalArgumentException("No reservation found for reservation number: " + reservationNumber)
reservation.complete()

public void cancelReservation(String accountNumber, String reservationNumber)
Validate accountNumber and reservationNumber are not null or empty (throw IllegalArgumentException if invalid)
Account account = getAccount(accountNumber)
If account is null:
Throw IllegalArgumentException("No account found for account number: " + accountNumber)
Reservation reservation = account.getReservation(reservationNumber)

If reservation is null:
Throw IllegalArgumentException("No reservation found for reservation number: " + reservationNumber)
reservation.cancel()

public void updateReservation(String accountNumber, Reservation reservation)
Validate accountNumber is not null or empty (throw IllegalArgumentException if invalid)
Validate reservation is not null (throw IllegalArgumentException if null)
Account account = getAccount(accountNumber)
If account is null:
Throw IllegalArgumentException("No account found for account number: " + accountNumber)
Reservation existing = account.getReservation(reservation.getReservationNumber())
If existing is null:
Throw IllegalArgumentException("No reservation found for reservation number: " + reservation.getReservationNumber())
existing.update()
Update fields on existing using setter methods based on values from reservation parameter

public double calculatePricePerNight(Reservation reservation)
Validate reservation is not null (throw IllegalArgumentException if null)
Return reservation.calculatePricePerNight()

public double calculateTotalReservationPrice(Reservation reservation)
Validate reservation is not null (throw IllegalArgumentException if null)
Return reservation.calculateTotalPrice()

Class Name: Address
Class Description/Purpose: The Address class represents a physical or mailing address used within the system. It is used by both Account and Reservation objects to store location-related information. This class functions as a simple data container with getter and setter methods for each address field.
Class Modifiers: public
Class Inheritance: none
Class Attributes:

- string – String – street address (house number and street name)
- city – String – city name
- state – String – state abbreviation or full state name
- zip – String – zip or postal code
- country – String – country name

Exceptions Thrown: none – does not perform validation, does not throw exceptions

Class Constructors: public Address(String street, String city, String state, String zip, String country)

Assign street parameter to attribute street
Assign city parameter to attribute city
Assign state parameter to attribute state
Assign zip parameter to attribute zip
Assign country parameter to attribute country

Class Methods:

Public String toString()
Create a formatted string containing street, city, zip, and country
Return formatted string

GETTER METHODS

public String getStreet()
Return street

public String getCity()
Return city

public String getState()
Return state

public String getZip()
Return zip

public String getCountry()
Return country

SETTER METHODS

public void setStreet(String street)
Assign street parameter to attribute street

public void setCity(String city)
Assign city parameter to attribute city

public void setState(String state)
Assign state parameter to attribute state

public void setZip(String zip)
Assign zip parameter to attribute zip

public void setCountry(String country)
Assign country parameter to attribute country

Class Name: HotelReservation
Class Description/Purpose: The HotelReservation class represents a reservation for a hotel property. It extends the abstract Reservation class and provides hotel-specific pricing logic. The nightly rate is calculated based on the number of beds, bedrooms, bathrooms, square footage, and whether the room includes a kitchenette.
Class Modifiers: public
Class Inheritance: extends Reservation
Class Attributes:

hasKitchenette – boolean – Indicates whether the hotel room includes a kitchenette. This feature increases the nightly rate if true.

Exceptions Thrown: none
Class Constructors: There is one constructor for HotelReservation. It initializes the shared reservation attributes by calling the superclass constructor and then sets the hotel-specific attribute.

public HotelReservation(String reservationNumber, LocalDate startDate, int numberOfNights, int numberOfBeds, int numberOfBedrooms, int numberOfBathrooms, int squareFeet, Address physicalAddress, Address mailingAddress, boolean hasKitchenette)

Call superclass constructor using super(...) with shared reservation parameters

Assign hasKitchenette parameter to attribute hasKitchenette

Class Methods:

public double calculatePricePerNight()
This method calculates the nightly rate for a hotel reservation. The base rate is calculated using lodging size characteristics, and an additional charge is applied if the room includes a kitchenette.
Initialize baseRate to 0
Add (numberOfBeds * 20) to baseRate
Add (numberOfBedrooms * 15) to baseRate
Add (numberOfBathrooms * 10) to baseRate
Add (squareFeet * 0.05) to baseRate
If hasKitchenette is true add 25 to baseRate
Return baseRate

public boolean hasKitchenette()
return hasKitchenette

public void setHasKitchenette(Boolean hasKitchenette)
assign hasKitchenette parameter to attribute hasKitchenette

Class Name: CabinReservation
Class Description/Purpose: The CabinReservation class represents a reservation for a cabin property. It extends the abstract Reservation class and provides cabin-specific pricing logic. The nightly rate is calculated based on the lodging's size attributes and includes an additional charge if the cabin includes a loft.
Class Modifiers: public
Class Inheritance: extends Reservation
Class Attributes:

hasLoft – boolean – indicates whether the cabin includes a loft. If true, additional charge is added to nightly rate

Exceptions Thrown: none – lifecycle and validation exceptions handled in superclass Reservation
Class Constructors: There is one constructor for CabinReservation. It initializes shared reservation fields by calling the superclass constructor and then sets the cabin-specific attribute.

public CabinReservation(String reservationNumber, LocalDate startDate, int numberOfNights, int numberOfBeds, int numberOfBedrooms, int numberOfBathrooms, int squareFeet, Address physicalAddress, Address mailingAddress, boolean hasLoft)

Call superclass constructor using super(...) with shared reservation parameters

Assign hasLoft parameter to attribute hasLoft

Class Methods:

public double calculatePricePerNight()
This method calculates the nightly rate for a cabin reservation. The base rate is computed using lodging size characteristics, and an additional amount is added if the cabin includes a loft.
Initialize baseRate to 0
Add (numberOfBeds * 25) to baseRate
Add (numberOfBedrooms * 20) to baseRate
Add (numberOfBathrooms * 15) to baseRate

Add (squareFeet * 0.07) to baseRate
If hasLoft is true add 40 to baseRate
Return baseRate

public boolean hasLoft()
return hasLoft

public void setHasLoft(boolean hasLoft)
assign hasLoft parameter to to attribute hasLoft

Class Name: HouseReservation
Class Description/Purpose: The HouseReservation class represents a reservation for a house property. It extends the abstract Reservation class and defines pricing behavior specific to house rentals. The nightly rate is calculated based on lodging size attributes and includes an additional charge if the house contains a full kitchen.
Class Modifiers: public
Class Inheritance: extends Reservation
Class Attributes:

hasfullKitchen – boolean – indicates whether the house includes a full kitchen. If true, an additional charge is added to the nightly rate.

Exceptions Thrown: none – exceptions inherited from the reservation class
Class Constructors: There is one constructor for HouseReservation. It initializes shared reservation data through the superclass constructor and sets the house-specific attribute.

public HouseReservation(String reservationNumber, LocalDate startDate, int numberOfNights, int numberOfBeds, int numberOfBedrooms, int numberOfBathrooms, int squareFeet, Address physicalAddress, Address mailingAddress, boolean hasFullKitchen)

Call superclass constructor using super(...) with shared reservation parameters

Assign hasFullKitchen parameter to attribute hasFullKitchen

Class Methods:

public double calculatePricePerNight()
This method calculates the nightly rate for a house reservation. The base rate is computed using lodging size characteristics, and an additional charge is applied if the house includes a full kitchen.
Initialize baseRate to 0
Add (numberOfBeds * 30) to baseRate

Add (numberOfBedrooms * 25) to baseRate
Add (numberOfBathrooms * 20) to baseRate
Add (squareFeet * 0.10) to baseRate
If hasFullKitchen is true add 50 to baseRate
Return baseRate

public boolean hasFullKitchen()
return hasFullKitchen

public void setHasFullKitchen(boolean hasFullKitchen)
assign hasFullKitchen parameter to attribute hasFullKitchen

Class Name: ReservationStatus
Class Description/Purpose: The ReservationStatus enumeration defines the lifecycle states that a reservation can have within the system. It is used by the Reservation class to enforce valid state transitions and to determine when pricing and updates are allowed.
Class Modifiers: public enum
Class Inheritance: none
Enumeration Values:

DRAFT – initial state of a reservation when it is first created. A reservation in DRAFT status can be updated, completed, or cancelled.

COMPLETED – indicates that the reservation has been finalized. When a reservation is completed, the total price is calculated and stored. Completed reservations cannot be cancelled or updated.

CANCELLED – indicates that the reservation has been cancelled. Cancelled reservations cannot be completed or updated.

Exceptions Thrown: none
Class Constructors: none
Class Methods: none

Class Name: IllegalOperationException
Class Description/Purpose: The IllegalOperationException class represents an exception that is thrown when a reservation lifecycle rule is violated. It is used by the Reservation class to prevent invalid operations such as completing a cancelled reservation, cancelling a completed reservation, or updating a reservation that is not editable. This exception stores contextual information about the failed operation to provide a detailed error message.
Class Modifiers: public

Class Inheritance: extends RuntimeException
Class Attributes:

serialVersionUID – long (static final) – Serialization identifier
operation – String (final) – the name of the operation being attempted (complete, cancel, update)
accountNumber – String (final) – the account number associated with the reservation
reservationNumber – String (final) – the reservation number associated with the failed operation
reason – String (final) – detailed description of why the operation is invalid

Exceptions Thrown: none
Class Constructors: There is one constructor used to create a detailed exception message when an illegal reservation operation occurs.

public IllegalOperationException(String operation, String accountNumber, String reservationNumber, String reason)
Call superclass constructor with message:
"IllegalOperationException: Cannot '" + operation + "' reservation '" + reservationNumber + "' for account '" + accountNumber + "'. Reason: " + reason
Assign operation parameter to attribute operation
Assign accountNumber parameter to attribute accountNumber
Assign reservationNumber parameter to attribute reservationNumber
Assign reason parameter to attribute reason

Class Methods:

public String toString()
return getMessage

GETTER METHODS

public String getOperation()
Return operation

public String getAccountNumber()
Return accountNumber

public String getReservationNumber()
Return reservationNumber

public String getReason()
Return reason

Class Name: DuplicateObjectException
Class Description/Purpose: The DuplicateObjectException class represents an exception that is thrown when an attempt is made to add an object that already exists in the system. It is used when adding duplicate accounts or duplicate reservations within an account. The exception generates a clear error message that identifies which object caused the duplication error.
Class Modifiers: public
Class Inheritance: extends RuntimeException
Class Attributes:

serialVersionUID – long (static final) – Serialization identifier
accountNumber – String (final) – the account number involved in the duplicate operation
reservationNumber – String (final) – the reservation number involved in the duplicate operation if applicable

Exceptions Thrown: none
Class Constructors: Depending on the context, this exception may be constructed with either only an account number (duplicate account) or both an account number and reservation number (duplicate reservation).

public DuplicateObjectException(String accountNumber)
Call superclass constructor with message:
"DuplicateObjectException: Account with number '" + accountNumber + "' already exists."
Assign accountNumber parameter to attribute accountNumber
Set reservationNumber attribute to null

public DuplicateObjectException(String accountNumber, String reservationNumber)
Call superclass constructor with message:
"DuplicateObjectException: Reservation '" + reservationNumber + "' already exists for account '" + accountNumber + "'."
Assign accountNumber parameter to attribute accountNumber
Assign reservationNumber parameter to attribute reservationNumber

Class Methods:

public String toString()
Return getMessage()

GETTER METHODS

public String getAccountNumber()
Return accountNumber

public String getReservation

Class Name: IllegalLoadException
Class Description/Purpose: The IllegalLoadException class represents an exception that occurs when an account file or reservation file cannot be successfully loaded into the system. It is intended to be used during file loading operations to indicate that required data could not be read or parsed correctly. The exception includes contextual information about which account and file failed to load.
Class Modifiers: public
Class Inheritance: extends RuntimeException
Class Attributes:

serialVersionUID – long (static final) – serialization identifier
accountNumber – String (final) – the account being loaded when the failure occurred
filename – String (final) – the file that could not be loaded
loadType – String (final) – indicates what failed (“account” or “reservation”)

Exceptions Thrown: none
Class Constructors: There is one constructor used to create a detailed load failure message.

public IllegalLoadException(String accountNumber, String fileName, String loadType)

Call superclass constructor with message:
"IllegalLoadException: Failed to load " + loadType + " file '" + fileName + "' for account '" + accountNumber + "'."

Assign accountNumber parameter to attribute accountNumber
Assign fileName parameter to attribute fileName
Assign loadType parameter to attribute loadType

Class Methods:

public String toString()
return getMessage

GETTER METHODS

public String getAccountNumber()
return accountNumber

public String getFileName()
return fileName

public String getLoadType()
return loadType

6 HUMAN INTERFACE DESIGN

This section presents the proposed graphical user interface for the Accommodation Reservation Management System. The screens and workflows were developed as design mockups to illustrate how users would interact with the application's reservation management features. The current Java implementation focuses on the underlying object-oriented classes and business logic; the graphical user interface has not been implemented. These mockups serve as a foundation for future interface development.

6.1 Overview of User Interface

The user interface (UI) provides a simple workflow for managing accounts and reservations. The UI is designed so that the user never edits system data directly. Instead, the UI collects user input, sends requests to the Manager class, and displays confirmation messages or error messages based on the result.

The UI is organized into two main areas of functionality:

1. Account Management
2. Reservation Management

The UI supports the following major actions:

Proposed System Startup and File Loading –
In the proposed interface, the application initializes the system by creating an instance of the Manager class. The current implementation starts with an empty in-memory collection of accounts. Future development will introduce file-based persistence, allowing the Manager to load previously saved account and reservation data. The proposed interface will display an appropriate error message if a file-loading operation fails.

Account Management Workflow –
The user can create a new account by entering an account number and contact information (mailing address, email, and phone). When the user selects "Create Account," the UI calls Manager.addAccount(Account) and displays either:

- A success message confirming the account was added, or
- An error message if the account already exists or input is invalid.

The user can search for an existing account by entering an account number and

selecting “Search.” The UI calls Manager.getAccount(accountNumber) and displays the account details if found. If not found, the UI displays a “not found” message.

The user can update account contact information (mailing address, email, phone) while viewing an existing account. When the user selects “Save Account Changes,” the UI updates the account fields and calls Manager.updateAccount(accountNumber) to finalize the update.

Reservation Management Workflow –
Reservations are always managed under a selected account. Once an account is loaded in the UI, the user can view that account’s reservations in a list/table. Each reservation entry shows key details such as reservation number, reservation type, start date, number of nights, status, and price (if completed).

The user can add a reservation by selecting “Add Reservation” and choosing a reservation type:

- Hotel Reservation
- Cabin Reservation
- House Reservation

The UI then displays a reservation entry form where the user enters reservation details. The UI creates the appropriate reservation object and sends it to Manager.addReservation(accountNumber, reservation). The UI displays:

- A success message if the reservation is added, or
- An error message if it is a duplicate or invalid.

The user can edit a reservation by selecting it from the list and choosing “Update Reservation.” The UI allows editing only when the reservation is in an editable state (for example, DRAFT and with a valid start date). When the user selects “Save Reservation Changes,” the UI sends the updated reservation information to Manager.updateReservation(accountNumber, reservation). If the reservation violates update rules, the UI displays the error message produced by IllegalOperationException.

The user can complete a reservation by selecting a reservation and choosing “Complete Reservation.” The UI calls Manager.completeReservation(accountNumber, reservationNumber). On success, the UI updates the reservation’s status to COMPLETED and displays the total price. If the operation is not allowed, the UI displays the error message.

The user can cancel a reservation by selecting a reservation and choosing “Cancel Reservation.” The UI calls Manager.cancelReservation(accountNumber, reservationNumber). On success, the UI updates status to CANCELLED. If cancellation is not allowed, the UI displays the error message.

Pricing Display –

The UI displays nightly price calculations when requested, and displays total price after completion. Since total price is only finalized after completion, the UI shows total price as “0” or “Not Available” for reservations that are still in DRAFT or CANCELLED status.

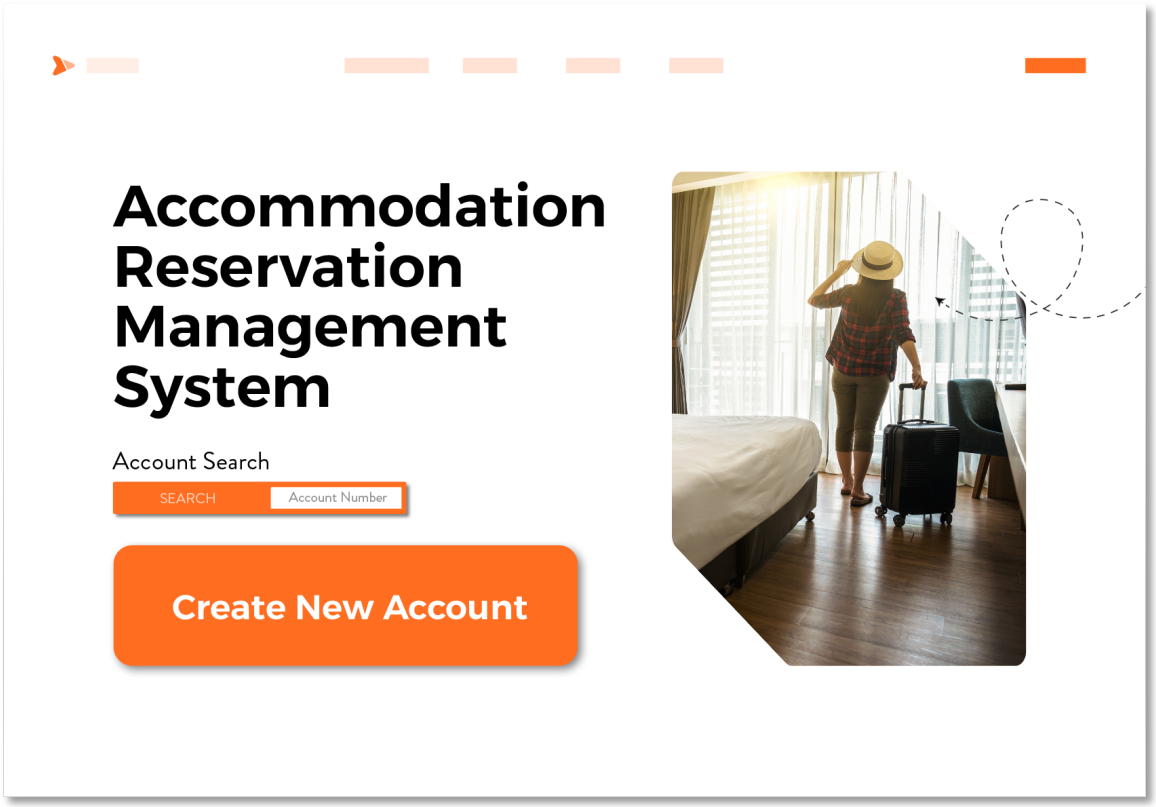
User Feedback and Error Handling –
For every action, the UI provides feedback to the user in the form of:

- Confirmation messages when operations succeed
- Clear error messages when operations fail

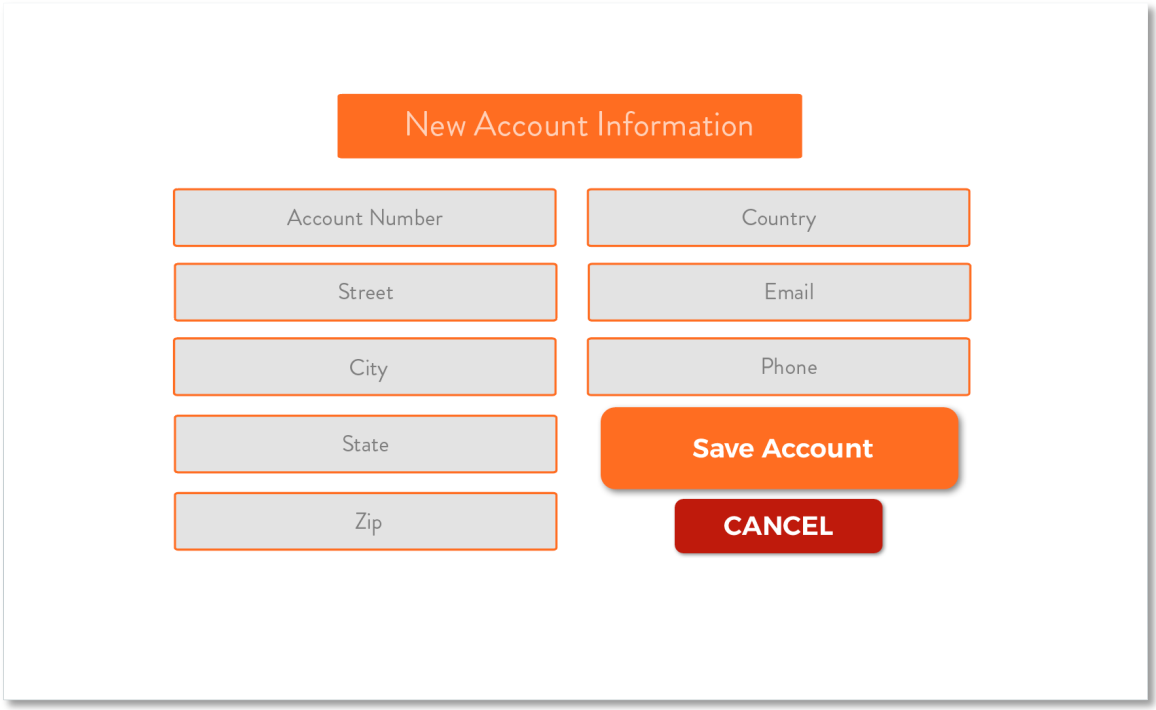
The implemented DuplicateObjectException and IllegalOperationException classes generate detailed error messages for duplicate objects and invalid reservation operations. The IllegalLoadException class is defined for future file-loading functionality and is not currently invoked by the application. The UI displays these messages as written to help the user understand what went wrong and what action was not allowed.

6.2 Screen Images

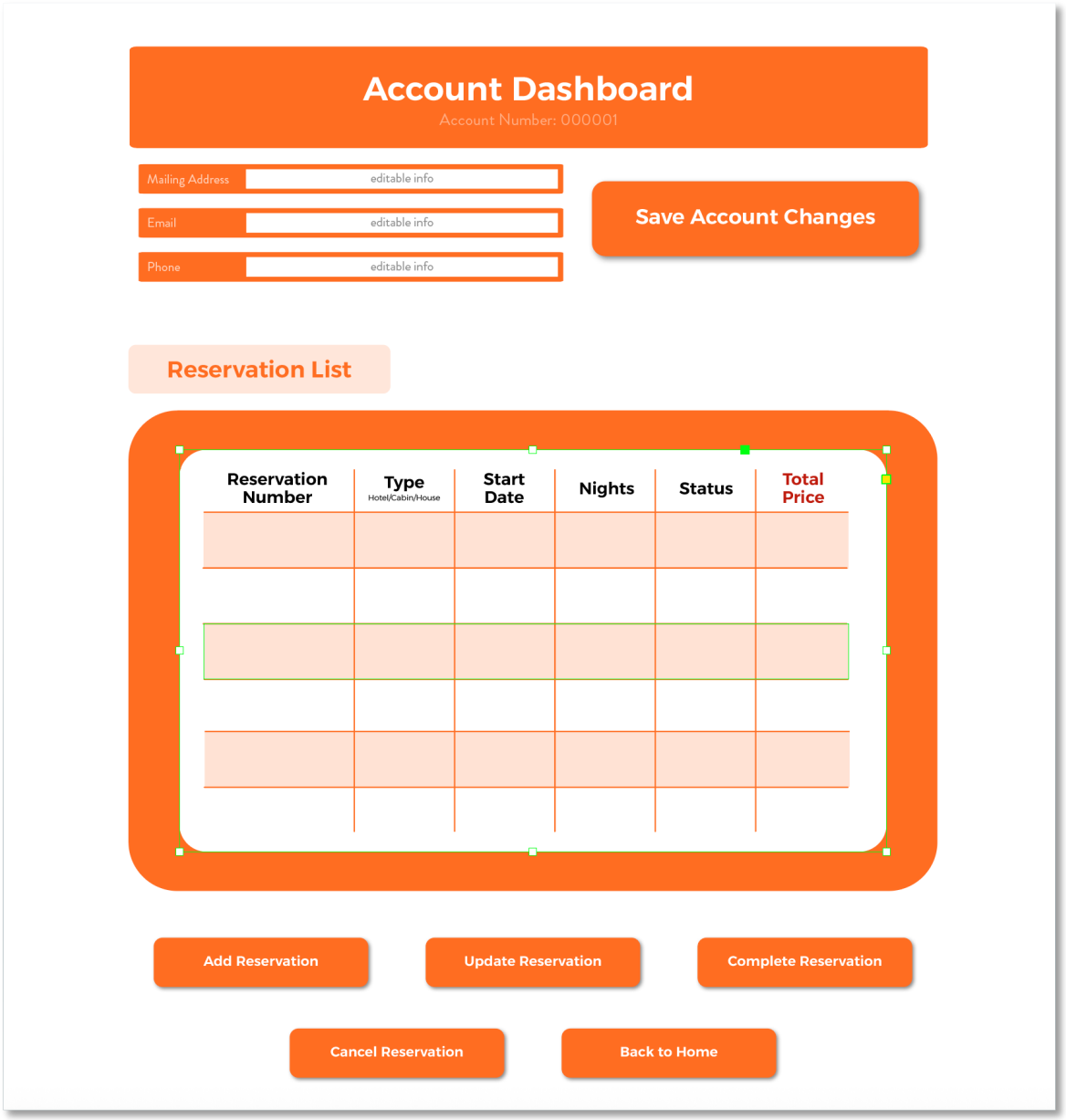
Home Screen –



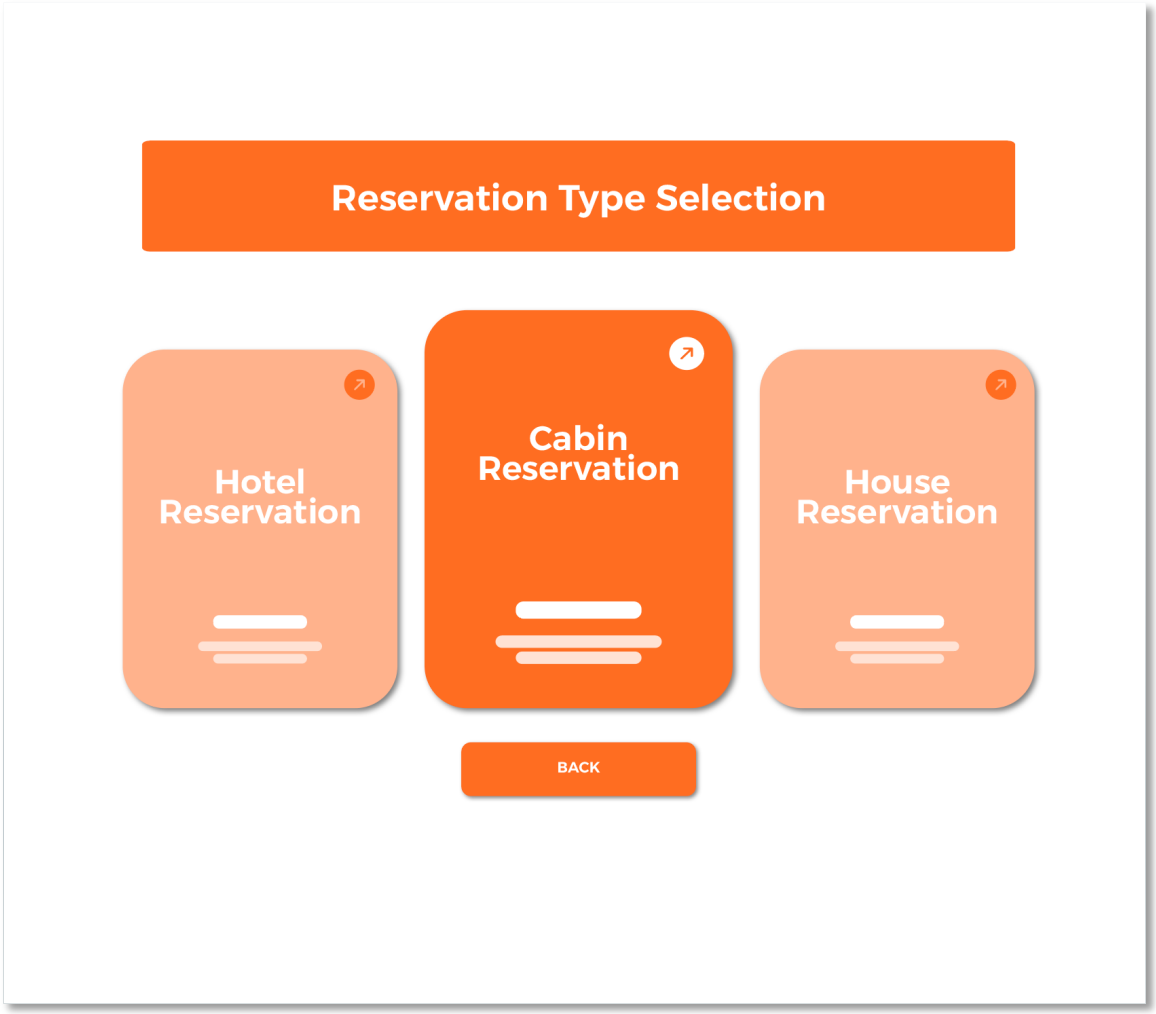
Create Account Screen –



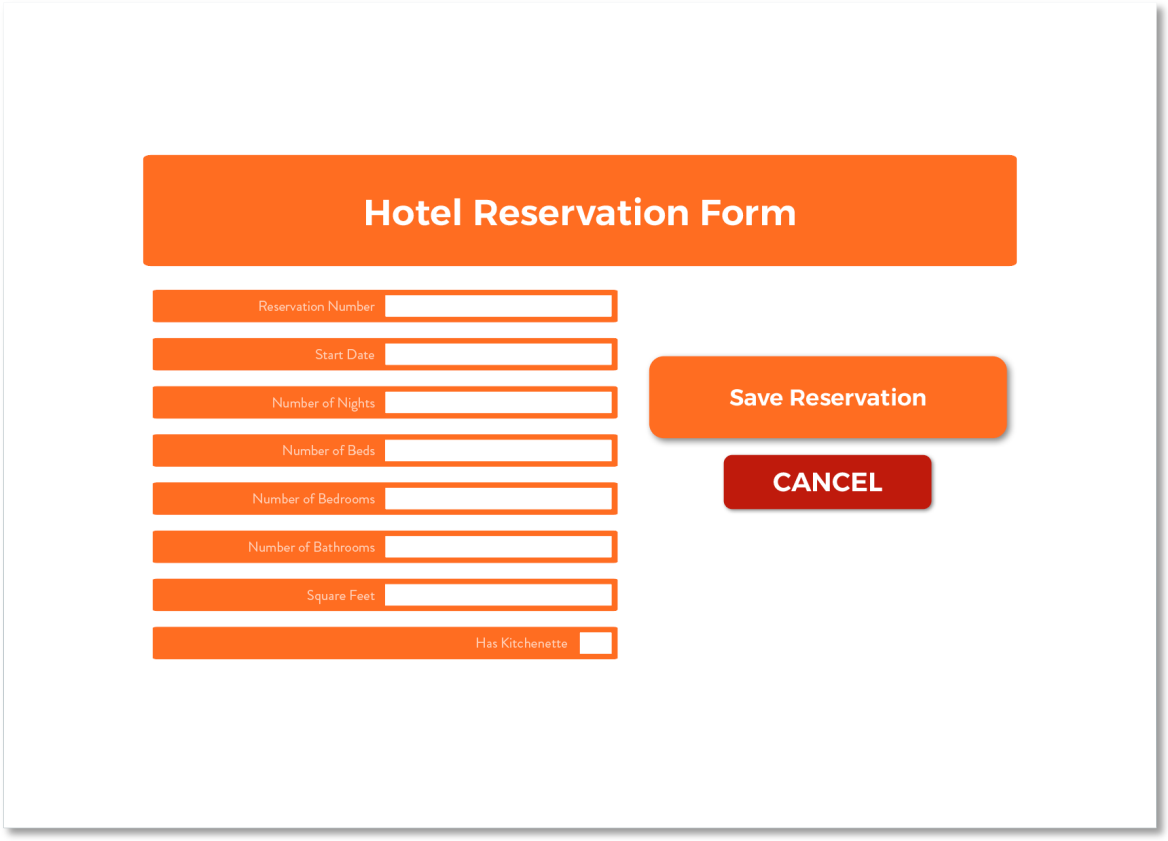
Account Dashboard Screen –



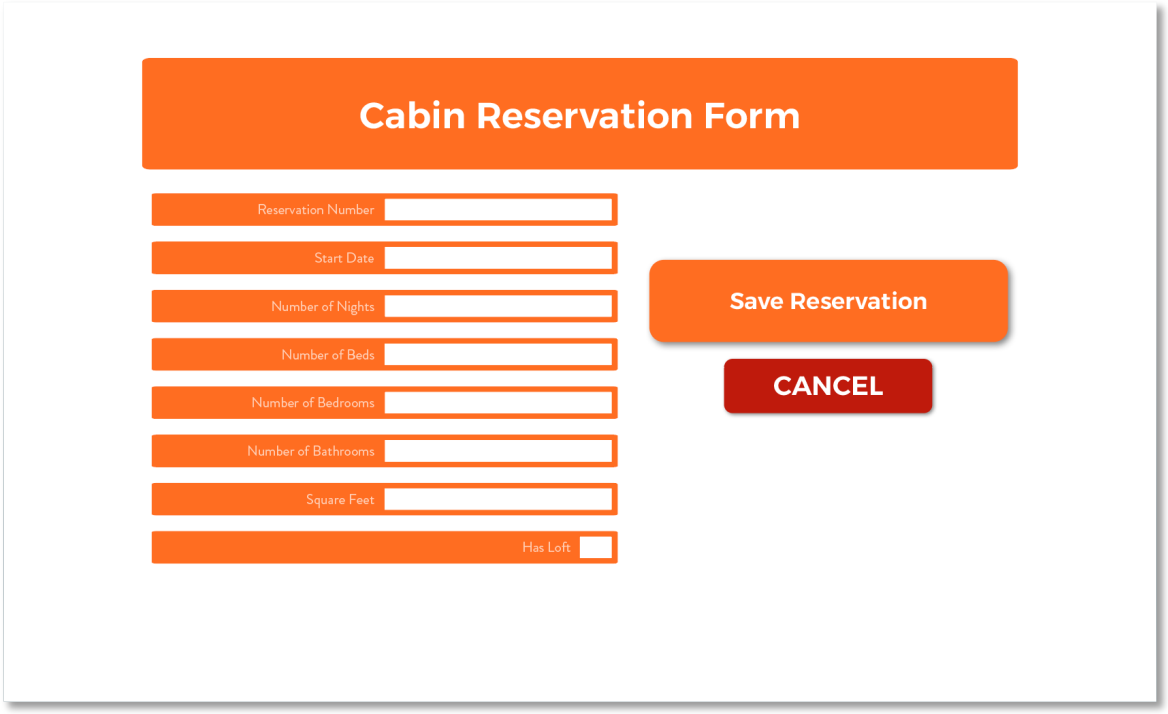
Reservation Type Selection Screen –



Reservation Form Screen 1 –



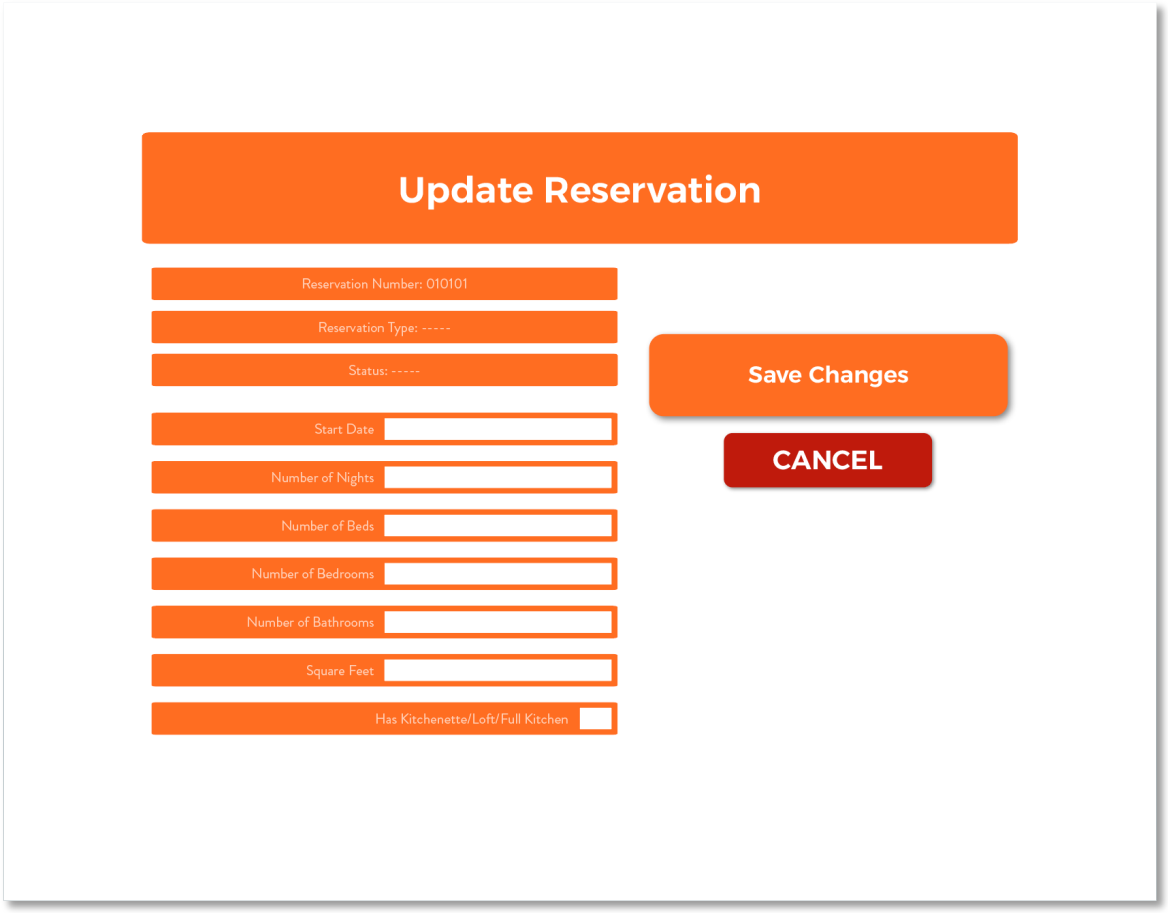
Reservation Form Screen 2 –



Reservation Form Screen 3 –



Update Reservation Screen –



6.3 Screen Objects and Actions

A discussion of screen objects and actions associated with those objects.

Home Screen Objects and Actions –

Account Number Text Field

- Allows user to enter an existing account number.

Search Account Button

- Calls Manager.getAccount(accountNumber)
- If account exists > Navigate to Account Dashboard
- If not found > Display error message

Create New Account Button

- Navigates to Create Account Screen

Create Account Screen Objects and Actions –

Input Fields
- Account Number
- Street
- City
- State
- Zip
- Country
- Email
- Phone

Save Account Button
- Creates Account object
- Calls Manager.addAccount(account)
- If successful > Navigate to Account Dashboard
- If duplicate > Display DuplicateObjectException message
- If invalid input > Display IllegalArgumentException message

Cancel Button
- Returns to Home Screen without saving

Account Dashboard Objects and Actions –

Editable Account Fields
- Mailing Address
- Email
- Phone

Save Account Changes Button
- Calls Manager.updateAccount(accountNumber)
- Displays confirmation message

Reservation Table
Displays:
- Reservation Number
- Reservation Type
- Start Date
- Number of Nights
- Status
- Total Price

Selecting a reservation enables reservation action buttons.

Add Reservation Button

- Navigates to Reservation Type Selection Screen

Update Reservation Button
- Requires reservation selection
- If reservation is editable (DRAFT and valid start date):
  -Navigate to Update Reservation Screen
- If not editable:
  -Display IllegalOperationException message

Complete Reservation Button
- Calls Manager.completeReservation(accountNumber, reservationNumber)
- If successful:
  -Updates reservation status to COMPLETED
  -Displays total price
- If invalid:
  -Displays IllegalOperationException message

Cancel Reservation Button
- Calls Manager.cancelReservation(accountNumber, reservationNumber)
- If successful:
  -Updates reservation status to CANCELLED
- If invalid:
  -Displays IllegalOperationException message

Back Button
- Returns to Home Screen

Reservation Type Selection Screen –

Hotel Reservation Button
- Navigates to Hotel Reservation Form

Cabin Reservation Button
- Navigates to Cabin Reservation Form

House Reservation Button
- Navigates to House Reservation Form

Back Button
- Returns to Account Dashboard

Reservation Form Screens (Hotel, Cabin, House) –

Common Input Fields
- Reservation Number
- Start Date
- Number of Nights
- Number of Beds
- Number of Bedrooms
- Number of Bathrooms
- Square Feet

Type Specific Field
- Hotel > Has Kitchenette
- Cabin > Has Loft
- House > Has Full Kitchen

Save Reservation Button
- Creates appropriate reservation subclass object
- Calls Manager.addReservation(accountNumber, reservation)
- If successful > Return to Account Dashboard
- If duplicate > Display DuplicateObjectException message
- If invalid > Display IllegalArgumentException message

Cancel Button
- Returns to Account Dashboard without saving

Update Reservation Screen –

Read-Only Fields
- Reservation Number
- Reservation Type
- Status

Editable Fields
- Start Date
- Number of Nights
- Number of Beds
- Number of Bedrooms
- Number of Bathrooms
- Square Feet
- Type-specific feature checkbox

Save Changes Button

- Calls Manager.updateReservation(accountNumber, updatedReservation)
- If successful > Return to Account Dashboard
- If invalid > Display IllegalOperationException message

Cancel Button

- Returns to Account Dashboard

7 REQUIREMENTS MATRIX

The following requirements traceability matrix maps the system's functional requirements to their corresponding Java classes, methods, and attributes. Most requirements are represented in the current implementation, while certain capabilities, including file-based persistence, remain part of the proposed design. The matrix documents both implemented functionality and planned enhancements to support future development.

Requirement	Class	Constructor / Method	Attribute
System will allow creation of a new account	Manager	addAccount(Account account)	accounts
Account must store unique account number	Account	Account(String, Address, String, String)	accountNumber
System will prevent duplicate account numbers	Manager	addAccount(Account account)	accountNumber
Account will store mailing address	Account	Account(...)	mailingAddress
Account will store email and phone	Account	Account(...)	email, phone
System will allow updating account contact information	Manager	updateAccount(String accountNumber)	mailingAddress, email, phone
Account will contain multiple reservations	Account	addReservation(Reservation reservation)	reservations
System will prevent duplicate reservations within an account	Account / Manager	addReservation(...)	reservationNumber
System shall support Hotel reservations	HotelReservation	HotelReservation(...)	hasKitchenette
System will support Cabin reservations	CabinReservation	CabinReservation(...)	hasLoft
System will support House reservations	HouseReservation	HouseReservation(...)	hasFullKitchen
Reservation will store start date	Reservation	Reservation(...)	startDate
Reservation will store number of nights	Reservation	Reservation(...)	numberOfNights
Reservation will store property size details	Reservation	Reservation(...)	numberOfBeds, numberOfBedrooms, numberOfBathrooms, squareFeet
Reservation will calculate nightly price based on type	Reservation subclasses	calculatePricePerNight()	type-specific fields
System will calculate total reservation price	Reservation	complete(), calculateTotalPrice()	price
Reservation will begin in DRAFT state	Reservation	Reservation(...)	status
System will allow reservation completion	Manager	completeReservation(String, String)	status
System will prevent completing cancelled reservation	Reservation	complete()	status
System will allow reservation cancellation	Manager	cancelReservation(String, String)	status
System will prevent cancelling completed reservation	Reservation	cancel()	status
System will prevent updating completed reservation	Reservation	update()	status
System will prevent updating cancelled reservation	Reservation	update()	status
System will enforce lifecycle states	ReservationStatus	enum values	DRAFT, COMPLETED, CANCELLED
System will throw exception for invalid lifecycle operation	IllegalOperationException	constructor	operation, reason
System will throw exception for duplicate objects	DuplicateObjectException	constructor	accountNumber, reservationNumber
System will load account and reservation data from files	Manager	Manager()	DATA_DIRECTORY
System will throw exception if file load fails	IllegalLoadException	constructor	fileName, loadType
System will allow viewing all reservations for an account	Account	getReservations()	reservations
System will allow searching for account	Manager	getAccount(String)	accountNumber
System will allow updating reservation details	Manager	updateReservation(String, Reservation)	reservation fields
Implementation Status: File-based persistence requirements, including loading account and reservation data and handling file-loading failures, are planned enhancements. The corresponding classes and attributes represent the proposed design and do not indicate completed functionality.			